Panduan Demo: Secure API dengan JWT

Cara mempresentasikan Task 01–03 di depan kelas memakai aplikasi **Creativo**: login dengan akun Creativo, token JWT, protected API, dan logout.

Durasi	Yang dibutuhkan	Yang dipakai di aplikasi
± 8 menit	HP dengan Expo Go (layarnya ditampilkan ke proyektor) dan laptop dengan terminal Metro	Layar **Akun Creativo** (Profil → kartu Akun Creativo), tab **Feed**, dan log **[api]** / **[auth]** di terminal

Satu kalimat yang harus diingat penonton

Login dengan akun Creativo menghasilkan **JWT**. JWT disimpan aman di **SecureStore**, dikirim di **setiap request** sebagai `Authorization: Bearer`, lalu **dihapus dan dicabut** saat logout. Tanpa token yang sah, server tidak memberi data.

1 Peta demo: tugas → yang ditunjukkan

Task (slide)	Perintah dosen	Yang ditunjukkan di depan	Bukti di terminal / test
Task 01 **Implement Token Authentication**	1. Login harus dapat JWT	**Masuk dengan Akun Creativo** → layar **Akun Creativo**: token, format **JWT · ES256**, peran, masa berlaku, **Disimpan di SecureStore** ✓	`[auth] Login berhasil → access token diterima`
Task 02 **Attach Token to API Request**	2. Token dipakai di setiap endpoint (login, 1 menu, logout)	Tab **Feed** dimuat ulang. Request tanpa token dan dengan token palsu dicoba dengan **curl**	`GET /rest/v1/posts → 200 (210 ms) · Bearer eyJ…` tanpa token → **[]**, token palsu → **401**
Task 03 **Protect the Session**	3. Token di-refresh saat logout	**Keluar dari Akun Creativo** → kembali ke layar awal → masuk lagi → **token dan masa berlaku baru**	`POST /auth/v1/logout → 204` `[auth] Logout → token dihapus`

Kenapa tidak memakai `mock-server.js` seperti contoh di kelas? Creativo sudah punya **server auth sungguhan**, yaitu Supabase Auth, yang mengeluarkan JWT. Prinsipnya sama persis: `POST login → token`, token dikirim di header `Authorization: Bearer`, lalu server memvalidasinya.

2 Persiapan (15 menit sebelum maju)

☐ **Supabase dashboard**	**Authentication** → **URL Configuration**: **Site URL** = `creativo://auth/callback` (bukan localhost). **Redirect URLs** berisi `creativo://**` dan `exp://**`.
☐ **Akun Creativo**	Akun sudah dibuat dan email sudah dikonfirmasi. Catat email dan kata sandinya. **Keluar dulu** sebelum maju, supaya demo dimulai dari keadaan belum login.
☐ **Jalankan aplikasi**	Di folder project: `npx expo start -c`. Buka Creativo di Expo Go. HP dan laptop di WiFi yang sama.
☐ **Layar HP ke proyektor**	iPhone: **QuickTime Player** → **File** → **New Movie Recording** → pilih iPhone sebagai kamera, atau **iPhone Mirroring** di macOS. Android: `scrcpy`.

☐ **Terminal Metro**	Perbesar huruf (**Cmd +**, 3–4 kali) dan bersihkan layar (**Cmd K**) tepat sebelum mulai.
☐ **Terminal kedua**	Buka tab baru di folder project. Siapkan perintah **curl** dari Langkah 6, dan jalankan `npm test` sekali untuk memastikan semuanya lolos.
☐ **Internet**	Pastikan stabil. Siapkan hotspot HP sebagai cadangan.

3 Naskah demo langkah demi langkah

Setiap langkah berisi **apa yang dilakukan**, **apa yang ditunjukkan** (layar HP atau terminal), dan **contoh kalimat** yang bisa diucapkan. Warna menandai tugasnya: **Task 01**, **Task 02**, **Task 03**.

PEMBUKAAN · ± 30 detik

"Minggu lalu Creativo mengambil data dari API. Minggu ini API-nya diamankan, karena API yang bisa diakses belum tentu boleh diakses siapa saja. Creativo memakai token-based authentication dengan JWT dari Supabase Auth. Saya akan tunjukkan tiga hal sesuai tugas: login mendapatkan JWT, token dipakai di setiap endpoint, dan token di-refresh saat logout."

Langkah 1 · Run: belum ada token

Task 01 · ± 30 detik

LAKUKAN	Buka Creativo di HP. Arahkan perhatian ke layar awal, lalu ke terminal.
TUNJUKKAN	Layar awal: **Lanjutkan dengan Google**, **Masuk dengan Akun Creativo**, **Buat Akun Creativo**.

```
[auth] Belum login (tidak ada token)
```

UCAPKAN

"Aplikasi baru dibuka dan belum ada token di perangkat, jadi yang muncul adalah layar awal. Di terminal juga tertulis: belum login, tidak ada token."

Langkah 2 · Login dengan akun Creativo

Task 01 · ± 1 menit

LAKUKAN	Tekan **Masuk dengan Akun Creativo** → isi email dan kata sandi → **Masuk**.
TUNJUKKAN	Aplikasi terbuka (Beranda). Di terminal muncul request login dan token yang diterima.

```
[api] POST /auth/v1/token?grant_type=password → 200 (240 ms) · Bearer sb_publish…
      (kunci publik aplikasi, bukan token pengguna)
[auth] Login berhasil → access token diterima: Bearer eyJhbGciOi…X9fQ3k
      (JWT ES256 · role authenticated · berlaku 60 menit lagi), disimpan di SecureStore (terenkripsi)
```

UCAPKAN

"Aplikasi mengirim email dan kata sandi ke endpoint login. Request login sendiri belum membawa token pengguna, karena login-lah yang menghasilkan token. Server membalas 200 dengan access token. Awalannya eyJ, itu ciri JWT: eyJ adalah tanda kurung kurawal pembuka dalam base64."

Langkah 3 · Tunjukkan JWT-nya

Task 01 · ± 1,5 menit

LAKUKAN	Buka tab **Profil** → kartu **Akun Creativo**.
TUNJUKKAN	Status **Sedang masuk**. Bagian **Sesi login (JWT)**: **Token akses** eyJhbGciOi…X9fQ3k, **Format** JWT · ES256, **Peran** authenticated, **Berlaku s/d** jam (± 60 menit lagi).

UCAPKAN

"JWT terdiri dari tiga bagian: header, payload, dan signature. Dari header kita tahu algoritmanya ES256. Dari payload kita tahu perannya authenticated dan kapan token kedaluwarsa, kira-kira satu jam lagi. Signature dibuat server dengan kunci rahasia, jadi kalau isinya diubah sedikit saja, server akan menolak. Token sengaja tidak ditampilkan utuh, karena ini bearer token: siapa pun yang memegangnya dianggap pemiliknya."

Langkah 4 · Token tersimpan aman

Task 01 · ± 30 detik

LAKUKAN	Tetap di layar Akun Creativo. Tunjuk baris **Disimpan di**.
TUNJUKKAN	**SecureStore (terenkripsi) ✓**

UCAPKAN

"Tanda centang ini bukan hiasan. Aplikasi membaca ulang token dari SecureStore, mendekripsinya dengan kunci dari Keychain di iPhone atau Keystore di Android, lalu mencocokkannya dengan token yang sedang dipakai. Ini fitur secure storage dari Pekan 3 yang tetap dipakai."

Langkah 5 · Token dipakai di menu Feed

Task 02 · ± 1 menit

LAKUKAN	Buka tab **Feed**, lalu tarik layar ke bawah untuk memuat ulang.
TUNJUKKAN	Karya-karya tampil. Di terminal, request ke `/rest/v1/posts` membawa JWT yang sama.

```
[api] GET /rest/v1/posts?… → 200 (210 ms) · Bearer eyJhbGciOi…X9fQ3k
      (JWT ES256 · role authenticated · berlaku 59 menit lagi)
```

UCAPKAN

"Sekarang menu Feed. Setiap request otomatis membawa header Authorization: Bearer berisi JWT tadi. Yang menempelkannya adalah client Supabase, perannya sama dengan interceptor di slide. Server memeriksa token itu dulu, baru memberikan data karya."

Langkah 6 · Bukti: tanpa token ditolak

Task 02 · ± 1 menit

LAKUKAN	Pindah ke **terminal kedua**, lalu jalankan dua perintah di bawah ini satu per satu.
TUNJUKKAN	Perintah 1 (tanpa token) → **[]**. Perintah 2 (token palsu) → **HTTP 401** `"JWT cryptographic operation failed"`.

```
set -a; source .env; set +a
# 1. Tanpa token pengguna
curl -s "$EXPO_PUBLIC_SUPABASE_URL/rest/v1/posts?select=id&limit=1" \
  -H "apikey: $EXPO_PUBLIC_SUPABASE_ANON_KEY"
# 2. Token palsu
curl -s -w "\nHTTP %{http_code}\n" "$EXPO_PUBLIC_SUPABASE_URL/rest/v1/posts?select=id&limit=1" \
  -H "apikey: $EXPO_PUBLIC_SUPABASE_ANON_KEY" \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJyb2xlIjoiYXV0aGVudGljYXRlZCJ9.palsu"
```

UCAPKAN

“Endpoint yang sama, tetapi tanpa token pengguna: datanya kosong, karena aturan RLS di database hanya memberi data ke pengguna authenticated. Dengan token palsu, server menolak dengan 401 karena signature-nya tidak cocok. Jadi data hanya bisa diambil lewat protected API dengan token yang sah.”

Langkah 7 · Logout

Task 03 · ± 1 menit

LAKUKAN	Kembali ke HP: Profil → **Akun Creativo** → **Keluar dari Akun Creativo** → **Keluar**.
TUNJUKKAN	Aplikasi kembali ke layar awal. Di terminal muncul request logout dan token dihapus.

```
[api] POST /auth/v1/logout?scope=global → 204 (180 ms) · Bearer eyJhbGciOi…X9fQ3k (JWT ES256 · …)
[auth] Logout → token dihapus dari SecureStore (terenkripsi), sesi berakhir
```

UCAPKAN

“Logout juga endpoint yang memakai token. Server mencabut refresh token di semua perangkat, lalu aplikasi menghapus token dari SecureStore dan kembali ke layar awal.”

Langkah 8 · Masuk lagi: token baru

Task 03 · ± 1 menit

LAKUKAN	Masuk lagi dengan akun yang sama → Profil → **Akun Creativo**.
TUNJUKKAN	**Akhiran token** (6 karakter terakhir) dan **Berlaku s/d** berbeda dari sebelumnya.

UCAPKAN

“Setelah masuk lagi, server membuat token baru. Akhirannya berbeda, dan masa berlakunya dihitung mulai sekarang. Sesi lama tidak bisa diperpanjang lagi, karena refresh token-nya sudah dicabut saat logout. Itulah maksud token di-refresh ketika logout.”

Langkah 9 · Test otomatis

Bukti tambahan · ± 30 detik

LAKUKAN	Di terminal kedua: `npm test`.
TUNJUKKAN	Kelompok test **1. Login → dapat JWT**, **2. Token dipakai setiap mengakses endpoint**, **3. Logout → token dihapus & diganti token baru**, dan baris terakhir **Tests: 66 passed**.

```
Tests:       66 passed, 66 total
```

UCAPKAN

“Ketiga perintah tugas juga dibuktikan dengan test otomatis, termasuk token palsu, token kedaluwarsa, dan logout saat tidak ada internet.”

PENUTUP · ± 20 detik

“Jadi, login dengan akun Creativo menghasilkan JWT. JWT disimpan aman dan dikirim di setiap request. Tanpa token, server tidak memberi data. Dan saat logout, token dihapus dan dicabut, sehingga login berikutnya mendapat token baru. Terima kasih.”

4 Menjelaskan JWT dengan sederhana

Kalau ditanya “JWT itu apa?”, jelaskan tiga bagiannya, lalu pakai perumpamaan gelang tiket konser.

```
eyJhbGciOiJFUzI1NiIs…   .   eyJzdWIiOiI4ZjJj…        .   …X9fQ3k
        HEADER                    PAYLOAD                  SIGNATURE
{ "alg": "ES256",          { "sub": "8f2c…",            dibuat server dengan
  "typ": "JWT" }             "role": "authenticated",    kunci rahasia; kalau
                             "exp": 1792… }              isi diubah → ditolak
```

Bagian JWT	Gelang tiket konser	Di layar Akun Creativo
Header	Jenis gelang dan cara segelnya dibuat	**Format**: JWT · ES256
Payload	Nama pemilik dan jam berlakunya	**Peran**: authenticated · **Berlaku s/d**: jam
Signature	Segel hologram yang hanya bisa dibuat panitia. Penjaga pintu cukup mengecek segelnya, tidak perlu menelepon kantor (stateless)	Diperiksa oleh **server**, bukan oleh aplikasi
Bearer	Siapa pun yang memakai gelang boleh masuk, jadi gelang harus dijaga	Token disamarkan, disimpan di SecureStore, dan hanya dikirim lewat HTTPS
Logout	Gelang dirobek, kartu perpanjangannya dibatalkan	Token dihapus, refresh token dicabut

5 Jawaban pertanyaan demo (slide)

Pertanyaan	Jawaban singkat
1. Dari mana access token diperoleh?	Dari **Supabase Auth**, pada response `POST /auth/v1/token?grant_type=password` setelah email dan kata sandi akun Creativo benar. Isinya JWT (access token) dan refresh token.
2. Di mana token disimpan?	Di perangkat, **terenkripsi AES-256**. Ciphertext di AsyncStorage, kunci enkripsinya di **SecureStore** (Keychain / Keystore). Dibuktikan oleh baris “Disimpan di … ✓”.
3. Bagaimana token dikirim ke API?	Lewat header `Authorization: Bearer <token>`, ditambahkan otomatis oleh client Supabase di setiap request, lewat **HTTPS**.
4. Apa fungsi Bearer token?	Membuktikan bahwa request berasal dari pengguna yang sudah login. Siapa pun yang membawa token dianggap pemiliknya, karena itu token harus dijaga.
5. Apa yang terjadi jika token tidak dikirim?	Request dianggap **anonim**. RLS tidak memberi data (**[]**). Token palsu atau kedaluwarsa ditolak dengan **401**.
6. Apa yang terjadi ketika logout?	Refresh token **dicabut di server**, token **dihapus dari SecureStore**, sesi dibersihkan, aplikasi kembali ke layar awal. Login berikutnya mendapat **token baru**.

6 Pertanyaan tambahan yang mungkin ditanyakan

Pertanyaan	Jawaban singkat
Kenapa tidak memakai mock-server.js?	Creativo sudah punya server auth sungguhan (Supabase Auth). Alurnya sama: login → token → Bearer di header → server memvalidasi.

Pertanyaan	Jawaban singkat
Di mana token divalidasi?	Di **server**: signature dicek dengan kunci ES256 dan masa berlaku (`exp`) diperiksa. Aplikasi hanya membaca isinya untuk ditampilkan.
Apa bedanya access token dan refresh token?	Access token dipakai di setiap request dan berlaku 1 jam. Refresh token dipakai untuk meminta access token baru tanpa login ulang, dan itulah yang dicabut saat logout.
Kenapa request login tidak membawa JWT?	Karena token belum ada. Login hanya membawa kunci publik aplikasi (publishable key) yang menandai aplikasinya, bukan penggunanya.
Kalau token dicuri bagaimana?	Token disimpan terenkripsi dan hanya dikirim lewat HTTPS. Masa berlakunya pendek (maksimal 1 jam), dan logout mencabut refresh token. Jujur saja: access token yang sudah telanjur dicuri tetap berlaku sampai kedaluwarsa, karena JWT bersifat stateless.
Kenapa token tidak ditampilkan utuh?	Karena ini bearer token: siapa pun yang melihatnya bisa memakainya. Yang ditampilkan hanya 10 karakter awal dan 6 karakter akhir.

7 Kalau ada masalah saat demo

Gejala	Solusi cepat
Browser membuka localhost: “couldn’t connect to the server”	**Site URL** masih localhost: ubah di dashboard (lihat Persiapan). Email sebenarnya sudah terkonfirmasi, jadi kembali ke aplikasi dan masuk.
Terminal: `gagal, server tidak bisa dihubungi`	Cek internet atau pindah ke hotspot. Pastikan project Supabase aktif di dashboard.
Login gagal: “Konfirmasi alamat email kamu…”	Buka email, klik link konfirmasi, lalu masuk lagi.
Aplikasi masih memakai alamat lama	Hentikan Metro, lalu jalankan `npx expo start -c`.
Internet mati total	Tunjukkan `npm test` (berjalan tanpa internet) dan penjelasan di **week5.md**.